NeoSOFT Technologies

DevOps Engineer — Interview Q&A Guide

Role: DevOps Engineer (3+ Years Experience) | Prepared by Sanket Chopade

Topics: AWS · Kubernetes · Terraform · CI/CD · Production Troubleshooting

AWS | Kubernetes | Terraform | CI/CD | Troubleshooting

SECTION 01 | General / Introduction

Q1

Tell me about yourself.

ANSWER

Structure your answer using the **Present** → **Past** → **Future** framework. Keep it under 2 minutes.

- **Present:** Fresher DevOps Engineer with ~9 months internship experience at Hisan Labs, working on AWS, Kubernetes, Terraform, CI/CD pipelines, and monitoring stacks.
- **Past:** B.Tech in Aeronautical Engineering (RTM Nagpur University, 2025). Transitioned to DevOps through a structured upskilling path. Built ApnaKart — a 10-microservice cloud-native e-commerce platform on AWS EKS.
- **Key Tech Stack:** AWS (EKS, EC2, RDS, S3, VPC, IAM, CloudFront, Route53, Lambda), Terraform, Jenkins, GitHub Actions, Docker, Kubernetes, Helm, Kustomize, Datadog, Prometheus, Grafana, SonarQube, Trivy.
- **Achievement highlight:** Reduced deployment time by 77%, achieved 99.9% uptime, and maintained zero critical CVEs in production.
- **Future:** Looking to join a team where I can own infrastructure end-to-end and grow towards SRE or Platform Engineering.

Tip: Tie your ApnaKart project into the narrative — it's your strongest proof of hands-on work.

SECTION 02 | Amazon Web Services (AWS)

Q2

What are policies in Auto Scaling?

ANSWER

Auto Scaling Policies define **when and how** the Auto Scaling Group (ASG) adds or removes EC2 instances.

- **Target Tracking Policy:** Automatically adjusts capacity to keep a specific metric (e.g., CPU at 60%). Simplest to configure — AWS manages scale-in/out.
- **Step Scaling Policy:** Scales in defined steps based on CloudWatch alarm thresholds. E.g., add 2 instances if CPU > 70%, add 4 if CPU > 90%.
- **Simple Scaling Policy:** Legacy. Triggers one action per alarm breach, then waits for a cooldown period before re-evaluating. Less responsive.
- **Scheduled Scaling:** Scales at predefined times. Useful for predictable traffic patterns (e.g., business hours, batch jobs).
- **Predictive Scaling:** Uses ML to forecast load and pre-emptively scales. Best for recurring traffic patterns.
- **Cooldown Period:** Time (seconds) after a scaling activity where no further scaling occurs. Prevents thrashing. Default: 300 seconds for simple policies.

Real-world note: For most web apps, Target Tracking with CPUUtilization or ALBRequestCountPerTarget is the go-to choice.

Q3

How do path-based routing and host-based routing work in ALB, and which has priority?

ANSWER

Application Load Balancer (ALB) supports **content-based routing** using Listener Rules evaluated in priority order.

- **Path-Based Routing:** Routes based on the URL path. E.g., `/api/*` → API target group, `/images/*` → Static assets target group.
- **Host-Based Routing:** Routes based on the HTTP Host header. E.g., `app.example.com` → App TG, `api.example.com` → API TG.
- **Priority:** ALB Listener Rules are evaluated by **numeric priority (1 = highest)**. Whichever rule matches first (lowest number) wins — regardless of whether it's path-based or host-based.
- **Combined Rules:** You can combine both conditions in a single rule (AND logic). E.g., Host = `api.example.com` AND Path = `/v2/*`.
- **Default Rule:** Always last (no priority number). Acts as the catch-all fallback.

Key interview point: There is no inherent priority between path vs host-based — priority is determined by the rule number you assign, not the condition type.

Q4

How can you give SSH access to an EC2 instance without sharing the PEM key?

ANSWER

Multiple secure approaches exist — know at least 3:

- **AWS Systems Manager (SSM) Session Manager [RECOMMENDED]:** Connect via browser or CLI without SSH port or keys. Requires SSM Agent on instance and `AmazonSSMManagedInstanceCore` IAM role. No port 22 needed.
- **EC2 Instance Connect:** AWS pushes a temporary one-time public key to the instance for 60 seconds via the API. User uses their own key pair or the console. Requires IAM permission `ec2-instance-connect:SendSSHPublicKey`.
- **Bastion Host with User's Own Keys:** Add the user's public key to `~/.ssh/authorized_keys` on the instance. They use their private key — you never share the original PEM.
- **Ansible/Userdata:** During launch, inject a user's public key via EC2 userdata script into authorized_keys.
- **AWS IAM Identity Center + SSH:** Enterprise approach using SAML-based identity federation.

Best answer: SSM Session Manager — no open ports, full audit trail in CloudTrail, no keys to manage.

SECTION 03 | Terraform (Infrastructure as Code)

Q5

What is Terraform lifecycle?

ANSWER

The **lifecycle** meta-argument in Terraform controls how resources are created, updated, and destroyed.

- **create_before_destroy:** Creates the replacement resource before destroying the old one. Critical for zero-downtime replacements (e.g., ASG, RDS). Default is destroy-then-create.
- **prevent_destroy:** Blocks `terraform destroy` for that resource. Safeguard for production databases, S3 buckets, etc.
- **ignore_changes:** Tells Terraform to ignore drift on specific attributes. E.g., if an external process changes tags, Terraform won't try to revert them.
- **replace_triggered_by:** Forces resource replacement when a referenced resource or attribute changes. Added in Terraform 1.2.

```
lifecycle {
create_before_destroy = true
prevent_destroy = true
ignore_changes = [tags]
}
```

Common trap: prevent_destroy only blocks terraform destroy — if you remove the block and run apply, it will delete the resource.

Q6

Difference between count and for_each in Terraform?

ANSWER

Both are meta-arguments for creating multiple resource instances, but they behave very differently:

- **count:** Creates N identical resources indexed by integer (0, 1, 2...). If you remove an item from the middle of a list, Terraform re-indexes and destroys/recreates multiple resources.
- **for_each:** Iterates over a **map or set of strings**. Each instance is keyed by the map key or set value. Removing one item only affects that specific resource — no cascading re-creation.
- **When to use count:** When resources are truly identical and order doesn't matter (e.g., creating 3 identical worker nodes).
- **When to use for_each:** When resources have distinct configurations (e.g., multiple S3 buckets with different names, IAM users from a list).
- **Addressing:** count → `aws_instance.web[0]` | for_each → `aws_instance.web["prod"]`

Rule of thumb: Default to for_each. Use count only when you genuinely have N identical things.

Q7

What is a data block and how is it different from a dynamic block?

ANSWER

- **Data Block:** Fetches read-only information from existing infrastructure or external sources. It queries (not creates) a resource. E.g., fetch an existing VPC, AMI, or secret from AWS.

```
data "aws_ami" "ubuntu" {
most_recent = true
filter { name = "name" values = ["ubuntu/images/*"] }
}
```

- **Dynamic Block:** Generates repeated nested configuration blocks inside a resource. Used when a resource attribute accepts a list of sub-blocks (e.g., ingress rules in a security group).

```
dynamic "ingress" {
for_each = var.ports
content {
from_port = ingress.value
to_port = ingress.value
protocol = "tcp"
}
}
```

- **Key difference:** Data block = read external data into Terraform state. Dynamic block = generate repeated nested blocks within a resource definition. They solve completely different problems.

Q8

What are provisioners in Terraform?

ANSWER

Provisioners execute scripts or commands on a resource **after creation or before destruction**. They are a last resort — Terraform's official stance is to avoid them when possible.

- **local-exec:** Runs a command on the machine running Terraform. E.g., trigger an Ansible playbook after EC2 creation.
- **remote-exec:** SSHs into the resource and runs commands. Requires connection block with SSH credentials.
- **file:** Copies files or directories from local to the remote resource.
- **creation-time provisioner:** Runs on resource creation (default).
- **destroy-time provisioner (when = destroy):** Runs before resource is destroyed. E.g., deregister from a service registry.
- **Why avoid them:** They introduce imperative logic into declarative IaC, don't integrate with plan/state, and failures can leave resources in an unknown state. Prefer cloud-init, user-data, or configuration management tools instead.

Interview answer: Mention you know provisioners exist but prefer user_data or SSM for EC2 config — shows maturity.

SECTION 04 | CI/CD & Code Quality (SonarQube)

Q9

What are the parameters of SonarQube and how do you test an application?

ANSWER

Core Quality Metrics (parameters) SonarQube tracks:

- **Bugs:** Code that is definitively wrong or will likely cause runtime errors.
- **Vulnerabilities:** Security weaknesses exploitable by attackers (e.g., SQL injection, hardcoded credentials).
- **Code Smells:** Maintainability issues — not bugs, but confusing or hard-to-maintain code.
- **Coverage:** Percentage of code exercised by unit tests. Configured via JaCoCo (Java), Istanbul (JS), etc.
- **Duplications:** Percentage of duplicated code blocks.
- **Reliability Rating / Security Rating / Maintainability Rating:** A–E grades per category.
- **Quality Gate:** A pass/fail threshold. If any metric breaches the gate, the pipeline fails.

How to test an application with SonarQube (in CI/CD):

- 1. Run unit tests with coverage report generation (e.g., `mvn test jacoco:report`).
- 2. Run SonarScanner: `sonar-scanner -Dsonar.projectKey=myapp -Dsonar.sources=. -Dsonar.host.url=$SONAR_URL -Dsonar.login=$SONAR_TOKEN`
- 3. SonarQube analyzes code and coverage, publishes results to the dashboard.
- 4. Jenkins/GitHub Actions checks the Quality Gate status via Sonar webhook — fails the build if gate is not passed.

In ApnaKart: SonarQube was integrated into Jenkins pipeline with a Quality Gate check — any critical vulnerability or coverage below threshold blocked deployment to UAT.

SECTION 05 | Production Troubleshooting & Security

Q10

If production keys are exposed, how do you troubleshoot and resolve the issue?

ANSWER

This is an **incident response** question. Show a structured, calm, step-by-step approach:

- **Step 1 — Immediate containment (within minutes):** Revoke/deactivate the exposed keys immediately via IAM console or AWS CLI. Do not wait to investigate first — contain first.
- **Step 2 — Assess blast radius:** Check CloudTrail logs for any API calls made using those keys after exposure. Look for: new IAM users created, S3 data access, EC2 launches, unauthorized role assumptions.
- **Step 3 — Rotate credentials:** Generate new access keys, update all services/secrets managers using the old key (AWS Secrets Manager, Kubernetes Secrets, .env files).
- **Step 4 — Identify the source:** How were keys exposed? GitHub commit? Log file? Slack message? Use git log, grep, or secret scanning tools (e.g., truffleHog, GitGuardian).
- **Step 5 — Remediate the root cause:** Remove keys from code, add .gitignore rules, enable AWS GuardDuty, set up secret scanning in pre-commit hooks.
- **Step 6 — Post-incident review:** Document the timeline, impact, and corrective actions. Implement long-term fix: use IAM roles (not long-lived keys), use IRSA for pods, use Secrets Manager instead of env vars.

Long-term fix: Never use static IAM access keys in production. Use IAM Roles for EC2 and IRSA/OIDC for Kubernetes pods — zero long-lived credentials.

SECTION 06 | Kubernetes

Q11

How do you upgrade a Kubernetes cluster without downtime?

ANSWER

Cluster upgrades require upgrading **control plane first, then worker nodes**. Zero-downtime relies on proper rolling strategy.

- **Pre-upgrade checks:** Verify app compatibility with the new K8s version. Check deprecated APIs using `kubectl deprecations` or Pluto. Take etcd backup.
- **Control Plane upgrade (EKS managed):** In AWS EKS, upgrade the cluster version via console or `eksctl upgrade cluster --name <name> --version <ver>`. AWS handles HA control plane rolling upgrade.
- **Worker Node upgrade (Node Groups):** Create a new node group with updated AMI/version. Cordon and drain old nodes one at a time: `kubectl cordon <node>` then `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`
- **PodDisruptionBudgets (PDB):** Set PDBs on critical workloads to ensure minimum replicas stay running during node drain.
- **Verify pods reschedule:** After drain, pods move to new nodes. Verify with `kubectl get pods -o wide`.
- **Update add-ons:** After node upgrade, update CoreDNS, kube-proxy, AWS CNI (vpc-cni) to compatible versions.
- **Rollback plan:** Keep old node group available until fully validated.

One minor version at a time — Kubernetes does not support skipping versions (e.g., 1.27 → 1.28 → 1.29, not 1.27 → 1.29).

Q12

Which Kubernetes version have you used?

ANSWER

Be honest and specific. Typical answer for 2024–2025 work:

- Primarily worked with **Kubernetes 1.28 and 1.29** on AWS EKS during internship.
- Kept track of deprecation notices (e.g., removal of PodSecurityPolicy in 1.25, deprecation of old batch/v1beta1 APIs).
- Used `kubectl version` and EKS managed node groups for lifecycle management.

If you used a different version in your actual work, state that version. Don't over-claim. Interviewers may follow up with version-specific API changes.

Q13

What are liveness and readiness probes?

ANSWER

Both are health checks defined in pod spec, but they serve distinct purposes:

- **Liveness Probe:** Checks if the container is **alive**. If it fails, kubelet **restarts the container**. Use case: detect deadlocks or hung processes that don't crash but can't serve requests.
- **Readiness Probe:** Checks if the container is **ready to receive traffic**. If it fails, the pod is **removed from Service endpoints** — traffic stops routing to it. Use case: wait for app startup, DB connections, cache warm-up.
- **Startup Probe:** Delays liveness/readiness checks during slow startup. Prevents premature restarts for apps that take long to initialize.
- **Probe types:** HTTP GET (most common), TCP Socket, Exec (run command inside container).

```
livenessProbe:
httpGet:
path: /healthz
port: 8080
initialDelaySeconds: 15
periodSeconds: 10
readinessProbe:
httpGet:
path: /ready
port: 8080
initialDelaySeconds: 5
periodSeconds: 5
```

Key distinction: Liveness failure = restart container. Readiness failure = remove from load balancer. Both can be configured independently.

Q14

What are static pods and how do you handle them?

ANSWER

Static pods are managed **directly by kubelet** on a specific node — not by the API server or scheduler.

- **Definition:** Pod manifests placed in `/etc/kubernetes/manifests/` on a node. Kubelet watches this directory and automatically creates/restarts these pods.
- **Key characteristic:** They always run on the node where the manifest lives. The API server sees a mirror object (read-only) but cannot delete or reschedule them.
- **Use case:** Core Kubernetes control plane components are static pods — `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, `etcd` on kubeadm clusters.
- **Handling / Troubleshooting:** To modify: edit the manifest file in `/etc/kubernetes/manifests/` directly on the node. Kubelet auto-reloads. To delete: remove the file from that directory.
- **Identification:** Static pods have the node name appended to their pod name. E.g., `kube-apiserver-master-node`.

In EKS: control plane static pods are AWS-managed — you don't have direct access to the control plane nodes. This question is more relevant for self-managed/kubeadm clusters.

Q15

Explain Nginx pod structure.

ANSWER

An Nginx pod in Kubernetes typically follows this structure:

- **Pod:** Wraps one or more containers. In a basic Nginx setup, one container running the `nginx:alpine` image.
- **Container spec:** Defines image, ports (containerPort: 80), resource requests/limits, volume mounts, env vars, and probes.
- **ConfigMap:** Nginx configuration (`nginx.conf`) mounted as a volume at `/etc/nginx/nginx.conf`. Allows config changes without rebuilding image.
- **Volumes:** ConfigMap volume for nginx.conf, optional emptyDir for temporary cache, PVC if serving static files from persistent storage.
- **Liveness Probe:** HTTP GET on `/` port 80. Readiness probe same path.
- **Service:** ClusterIP (internal) or LoadBalancer/NodePort (external) exposes port 80 of the pod.
- **Sidecar pattern (advanced):** Sometimes a sidecar container (e.g., log shipper like Filebeat) runs alongside Nginx in the same pod, sharing the log volume.

```
containers:
- name: nginx
image: nginx:1.25-alpine
ports:
- containerPort: 80
volumeMounts:
- name: nginx-config
mountPath: /etc/nginx/nginx.conf
subPath: nginx.conf
```

Q16

Explain Kubernetes Deployment YAML file.

ANSWER

A Deployment YAML has 4 main sections:

- **apiVersion & kind:** `apiVersion: apps/v1`, `kind: Deployment`. Tells the API server what object type this is.
- **metadata:** Name, namespace, labels. Labels on the Deployment itself (not the pods).
- **spec.replicas:** Number of desired pod replicas. The ReplicaSet controller maintains this count.
- **spec.selector:** `matchLabels` — how the Deployment finds the pods it owns. Must match `template.metadata.labels`.
- **spec.strategy:** RollingUpdate (default) or Recreate. RollingUpdate uses `maxSurge` and `maxUnavailable` to control the upgrade pace.
- **spec.template:** Pod template. Defines the actual pod — metadata (labels that match selector), and spec (containers, volumes, probes, affinity, tolerations).
- **spec.template.spec.containers:** List of containers — image, ports, env, resources (requests/limits), volumeMounts, livenessProbe, readinessProbe.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp
namespace: production
spec:
replicas: 3
selector:
matchLabels:
app: myapp
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1
maxUnavailable: 0
template:
metadata:
labels:
app: myapp
spec:
containers:
- name: myapp
image: myapp:v1.2
ports:
- containerPort: 8080
resources:
requests:
cpu: 250m
memory: 256Mi
limits:
cpu: 500m
memory: 512Mi
```

maxUnavailable: 0 with maxSurge: 1 is the zero-downtime rolling update strategy — always keep all replicas running, add one extra during update.